

# PASSIVEGUARD AI

Complete Technical Project Report | SIH Prototype: AI-Based Threat Detection in Unidirectional Traffic

---

PASSIVEGUARD AI — COMPLETE TECHNICAL PROJECT REPORT

SIH Prototype: AI-Based Detection of Cyber Threats in Unidirectional IP Traffic

Location: C:\Users\Pradeep Kumar\OneDrive\Desktop\passiveguard-ai

## 1. EXECUTIVE SUMMARY

---

Project Name: PassiveGuard AI

Repository Path: C:\Users\Pradeep Kumar\OneDrive\Desktop\passiveguard-ai

Current Version: v0.1.0-scaffold / v1.0.0-hybrid

SIH Problem Statement: "AI-Based Detection of Cyber Threats in Unidirectional IP Traffic"

Problem Being Solved:

High-security enclaves—such as industrial control systems (ICS/SCADA), power grids, nuclear power plants, defense command centers, and air-gapped government infrastructure—frequently utilize hardware data diodes to establish unidirectional IP network links. A data diode physically restricts data transmission to a single direction (e.g., from an isolated operational technology enclave to an enterprise monitoring network). While this guarantees that hostile external entities cannot transmit malicious packets back into the high-security enclave, it severely complicates network security monitoring:

**1. Security monitoring tools inside or outside the enclave receive only unidirectional traffic metadata.**

---

**2. Active probing, port scanning, ping sweeps, and TCP SYN handshakes are physically impossible because no return path exists.**

---

**3. Deep Packet Inspection (DPI) and TLS decryption are prohibited or impossible due to encryption, high throughput demands, and zero key-sharing policies.**

---

## 4. Traditional Network Detection and Response (NDR) systems fail because they rely on bidirectional flow reconstruction, TCP session handshake tracking, active mitigation responses (TCP RST injection), and external cloud threat intelligence lookups.

### Core Objective & Proposed Solution:

PassiveGuard AI is a non-intrusive, zero-emission, passive Network Detection & Response (NDR) platform explicitly engineered for unidirectional IP traffic enclaves. The system passively consumes unidirectional flow telemetry or PCAP packet feeds, extracts rich statistical and temporal feature vectors, evaluates traffic across seven independent threat detectors, fuses findings using a multi-detector risk engine and Random Forest Machine Learning (ML) classifiers trained on real cybersecurity benchmark datasets (UNSW-NB15), and broadcasts real-time security alerts to a React-based SOC dashboard over WebSockets.

### Target Environment & Target Users:

- Target Environments: Critical National Infrastructure (CNI), SCADA/ICS networks, data diode enclaves, defense command enclaves, financial vault enclaves, and zero-trust air-gapped facilities.
- Target Users: Security Operations Center (SOC) analysts, Industrial Cybersecurity Engineers, Operational Technology (OT) Security Officers, and Enclave Security Administrators.

### Main Technical Innovation:

PassiveGuard AI introduces a Hybrid Dual-Layer Detection Engine:

- Layer A (Behavioral & Statistical Heuristics): Computes deterministic anomaly metrics (destination fan-out, temporal inter-arrival periodicity, Shannon entropy of subdomains, packet-size variance, directional byte asymmetry).
- Layer B (Local Machine Learning Inference): Runs local RandomForestClassifier inference on normalized 12-feature vectors without invoking network sockets, cloud APIs, or active polling.
- Hybrid Fusion Engine: Merges Layer A and Layer B scores safely ( $\text{Fused Score} = \max(\text{Stat\_Score}, 0.50 * \text{Stat\_Score} + 0.50 * \text{ML\_Score})$ ), preventing ML false negatives from suppressing critical statistical security signatures while suppressing false positives through temporal cooldown deduplication.

### Simple Judge Explanation:

"Imagine a security guard standing behind a one-way mirror in a high-security vault. The guard can see everything leaving the room, but cannot speak, shout back, or touch anything inside. PassiveGuard AI acts as the intelligent brain behind that one-way mirror. It observes every packet leaving the enclave, mathematically analyzes patterns without sending a single signal back, and warns the SOC team the instant an anomaly—such as a data leak, scanning attempt, or malware beacon—is detected."

### Current Implementation Status:

- Backend Architecture: FastAPI asynchronous application with SQLite alert store and WebSocket broadcasting (251/251 tests passing).
- Ingestion & Feature Engine: Read-only PCAP parser (scapy/dpkt), unidirectional flow aggregator, and 12-feature extractor.
- 7 Threat Detectors: Implemented for DDoS, C2 Beaconing, DGA, DNS Tunnelling, Encrypted Session Malware, Reconnaissance, and Data Exfiltration.

- ML Pipeline: Local dataset ingestion (UNSW-NB15), time-aware and predefined benchmark splitting, leakage-safe scaling, and Random Forest classifier training (ddos\_rf\_unsw\_nb15\_v1.joblib and recon\_scan\_rf\_unsw\_nb15\_v1.joblib).
- SOC Dashboard: React + Vite + Tailwind CSS + Recharts interactive dashboard displaying real-time alert streams, threat metrics, and detailed feature evidence.

## 2. PROBLEM STATEMENT ANALYSIS

---

What is Unidirectional IP Traffic?

Unidirectional IP traffic is network communication constrained physically or logically to transmit in exactly one direction (A -> B) with zero transmission in the reverse direction (B -> A). In a data diode deployment, physical light-emitting diodes (LEDs) and photoreceivers ensure that data flows from an enclave to an external network, but photons cannot travel backwards.

What is a Passive Monitoring Enclave?

A passive monitoring enclave is an isolated computing environment where security sensors observe data streams via passive network TAPs or data diode receiver ports. The sensor possesses no physical or logical ability to inject packets, establish TCP handshakes, issue ICMP probes, resolve DNS names, or execute active countermeasures.

Why Conventional Security Systems Fail:

**1. Dependency on Bidirectional TCP Handshakes: Traditional Intrusion Detection Systems track TCP state transitions (SYN -> SYN-ACK -> ACK). In unidirectional links, the monitoring sensor sees only SYN or SYN-DATA packets without corresponding SYN-ACK responses.**

---

**2. Strict Prohibition of Active Probing: Enterprise scanners emit active network probes to verify host liveness or service versions. In unidirectional enclaves, active probing is physically impossible and security policy strictly forbids any packet emission.**

---

**3. No Return-Path Communication: The sensor cannot query cloud threat intelligence feeds (e.g., VirusTotal, AbuseIPDB) or update signature databases dynamically over HTTP/REST APIs.**

---

**4. Encrypted Payload Blindness: Modern enterprise traffic utilizes TLS 1.3 and QUIC. Without private key possession or active MITM proxying, payload inspection is impossible.**

---

The Six Target Threat Categories in PassiveGuard AI:

**1. Volumetric / Protocol DDoS: Abnormally high packet arrival rate (>10,000 pkts/sec), SYN flag dominance without ACK responses, source entropy anomalies.**

---

**2. Botnet C2 Beacons: Periodic inter-arrival timestamps, recurring destination IP/port tuples, small uniform packet sizes.**

---

**3. DGA Domains: High Shannon entropy ( $H > 3.8$ ), high digit ratio, lexical randomness in DNS query metadata.**

---

**4. DNS Tunnelling: Long subdomain query strings (>30 chars), high query frequency to single apex domain, high byte volume over UDP port 53.**

---

**5. Encrypted TLS/QUIC Malware: Fixed TLS record length distributions, suspicious JA3/JA4 fingerprint hashes, SSLv3/legacy protocol usage.**

---

**6. Reconnaissance / Port Scanning: Single source IP connecting to many distinct ports (vertical) or dest IPs (horizontal) within short temporal windows.**

---

**7. Data Exfiltration: Unidirectional byte asymmetry ( $\text{Bytes\_out} \gg \text{Bytes\_in}$ ), sustained high-volume outbound TCP transfers to rare destinations.**

---

### 3. PROPOSED SOLUTION

---

PassiveGuard AI enforces an Observation-Only Pipeline Architecture. It reads streaming network flows or PCAP files, computes statistical metrics and ML features, evaluates threat detectors, fuses risks, and persists findings to a local SQLite database for SOC display.

Pipeline Flow:

Traffic Source -> Passive Ingestion -> Feature Engine -> 7 Threat Detectors + Layer B ML Engine -> Risk Fusion Engine -> Alert Store & Cooldown -> React SOC Analyst Dashboard

Core Operating Principles:

**1. Passive Observation Only: Zero packet creation, zero socket binding for network output, zero active probing.**

---

**2. Payload-Agnostic Metadata Analysis: Operates strictly on header metadata (IP, port, TCP flags, duration, packet sizes, DNS subdomain strings, TLS SNI/JA3). Zero TLS decryption required.**

---

**3. Explainable Evidence Generation: Every generated alert includes an evidence JSON dictionary explaining why the alert fired (e.g., exact Shannon entropy, fan-out count, sub-score breakdown, feature importances).**

---

### 4. SYSTEM ARCHITECTURE

---

Frontend: React 18 + Vite + Tailwind CSS + Recharts + Lucide React + Axios

Backend: FastAPI (Uvicorn ASGI, Lifespan Alert Poller Task @ 250ms)

Alert Store: SQLite (data/alerts.db) via App AlertStore

Pipeline Engine: PipelineEngine (orchestrates ingestion, features, detectors, fusion)

Risk Fusion: RiskFusionEngine (Corroboration, Cooldown, Primary Threat Selection)

Detectors: DDoS, C2, DGA, DNS Tunnelling, TLS Malware, Recon, Exfiltration

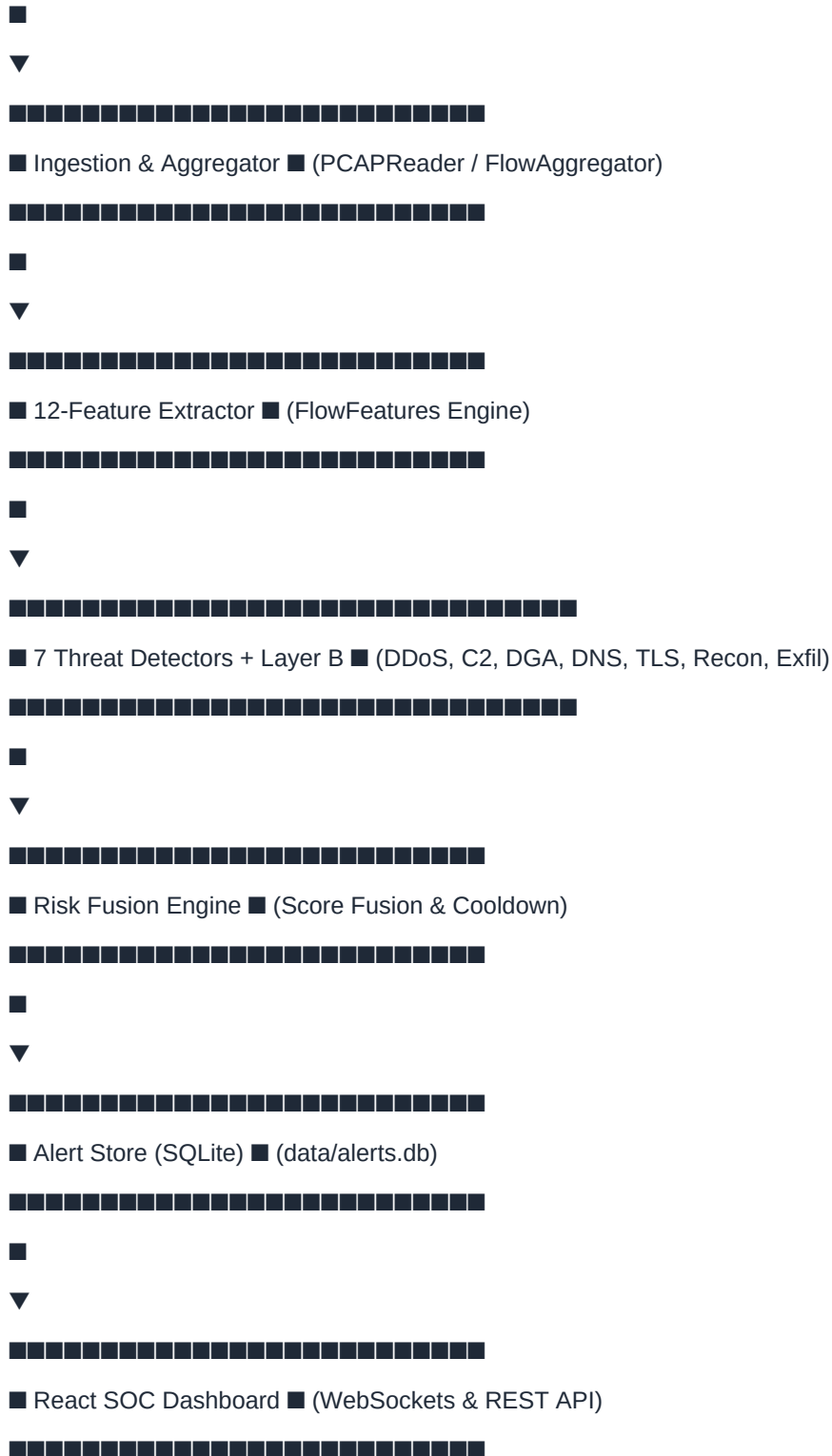
ML Inference Engine: MLInferenceEngine (Joblib Model Loading & StandardScaler)

Feature Extraction: FlowFeatures Engine (12-Feature Extraction)

### Ingestion Layer: PCAPReader & FlowStream (Read-only PCAP / Replay)

### ASCII Architecture Diagram:

## Passive / One-Way Traffic Stream



## 5. TECHNOLOGY STACK

---

### Frontend:

- Core Framework: React 18.2.0 (react, react-dom)
- Build Tooling: Vite 5.1.0 (@vitejs/plugin-react)
- Routing: React Router DOM 6.22.0
- Styling: Tailwind CSS 3.4.1 + Autoprefixer + PostCSS
- Data Visualization: Recharts 2.12.0
- Icons: Lucide React 0.330.0
- HTTP Client: Axios 1.6.7

### Backend:

- Language & Runtime: Python 3.14.6 (Compatible with Python 3.10+)
- Web Framework: FastAPI 0.110+
- ASGI Server: Uvicorn
- Data Validation: Pydantic v2 (BaseModel, ConfigDict, Field)
- Asynchronous Execution: Python asyncio Event Loop

### Machine Learning:

- ML Library: scikit-learn (1.4+)
- Classifiers: RandomForestClassifier, HistGradientBoostingClassifier
- Data Preprocessing: StandardScaler, pandas, numpy
- Model Serialization: joblib

### Data & Persistence:

- Database Engine: SQLite 3 (data/alerts.db)
- Packet Ingestion: scapy / dpkt read-only PCAP parsing
- Formats: JSON, CSV, PCAP, Joblib

### Testing Framework:

- Test Runner: pytest 9.1.1 (pytest-asyncio, anyio)
- HTTP Testing: FastAPI TestClient

## 6. PROJECT STRUCTURE

---

passiveguard-ai/

■■■ backend/

■ ■■■ app/

- ■ ■ ■ ■ config.py
- ■ ■ ■ ■ main.py
- ■ ■ ■ ■ alerts/ (manager.py, schema.py, store.py)
- ■ ■ ■ ■ api/ (alerts.py, models.py, traffic.py, websocket.py)
- ■ ■ ■ ■ detection/ (base.py, c2.py, ddos.py, dga.py, dns\_tunneling.py, exfiltration.py, recon.py, tls.py)
- ■ ■ ■ ■ features/ (flow\_features.py, models.py)
- ■ ■ ■ ■ ingestion/ (flow\_stream.py, models.py, pcap\_reader.py)
- ■ ■ ■ ■ ml/ (inference.py, preprocessing.py, trainer.py, datasets/)
- ■ ■ ■ ■ pipeline/ (engine.py)
- ■ ■ ■ ■ risk/ (fusion.py, scoring.py)
- ■ ■ ■ ■ tests/ (22 test files, 251 test cases)
- ■ ■ ■ data/
- ■ ■ ■ ■ alerts.db
- ■ ■ ■ ■ manifests/ (ddos\_rf\_unsw\_nb15\_v1.json, recon\_scan\_rf\_unsw\_nb15\_v1.json)
- ■ ■ ■ ■ models/ (ddos\_rf\_unsw\_nb15\_v1.joblib, recon\_scan\_rf\_unsw\_nb15\_v1.joblib, etc)
- ■ ■ ■ ■ docs/ (Documentation & Reports)
- ■ ■ ■ ■ frontend/ (React 18 + Vite SOC Dashboard)
- ■ ■ ■ ■ scripts/ (generate\_demo\_data.py, run\_demo.py, train\_models.py, validate\_real\_dataset.py)
- ■ ■ ■ ■ README.md

## 7. TRAFFIC INGESTION

PCAP File Reader: backend/app/ingestion/pcap\_reader.py (IMPLEMENTED - Offline Read-Only File Stream)

Streaming Flow Aggregator: backend/app/ingestion/flow\_stream.py (IMPLEMENTED - In-Memory Flow Aggregation)

Controlled Telemetry Replay: scripts/generate\_demo\_data.py (IMPLEMENTED - Memory-Buffered Simulation)

Live Network TAP Interface: backend/app/ingestion/ (SIMULATED / PLANNED - Hardware TAP / Data Diode)

How Packets Become Flow Records:

**1. Packet Capture (ObservedPacket): PCAPReader iterates through packet headers using scapy or dpkt, parsing IP addresses, ports, transport protocol, frame length, TCP flags, and microsecond timestamps.**

**2. Flow Aggregation (FlowAggregator):** Packets sharing the 5-tuple identifier (`src_ip`, `dst_ip`, `src_port`, `dst_port`, `protocol`) are grouped into a streaming `FlowRecord`.

---

**3. Timeout Eviction:** An inactive flow is finalized and emitted to the feature pipeline when its inter-packet inactivity gap exceeds `flow_timeout` (default: 15.0 seconds).

---

## 8. FEATURE ENGINEERING

---

12-Feature Vector Schema:

**1. `flow_duration`:** `t_last - t_first` (seconds)

---

**2. `flow_packet_count`:** Total packets observed in flow window

---

**3. `flow_byte_count`:** Total payload & header bytes

---

**4. `flow_packets_per_sec`:** `packet_count / max(duration, 0.001)`

---

**5. `flow_bytes_per_sec`:** `byte_count / max(duration, 0.001)`

---

**6. `tcp_syn_count`:** Total TCP packets carrying SYN flag

---

**7. `tcp_ack_count`:** Total TCP packets carrying ACK flag

---

---

## 8. tcp\_syn\_ack\_ratio: $\text{syn\_count} / \max(\text{ack\_count}, 1)$

---

## 9. directional\_byte\_ratio: $\text{bytes\_src\_to\_dst} / \max(\text{byte\_count}, 1)$

---

## 10. directional\_packet\_ratio: $\text{pkts\_src\_to\_dst} / \max(\text{packet\_count}, 1)$

---

## 11. packet\_size\_mean: $\text{byte\_count} / \max(\text{packet\_count}, 1)$

---

## 12. packet\_size\_std: Standard deviation of packet sizes

---

Protocol-Specific Feature Extraction:

- DNS Subdomain Shannon Entropy (H): Calculated over DNS query labels to detect DGA domains and covert DNS tunnelling.
- TLS Metadata Features: Evaluates `tls_version`, SNI hostname, `tls_ja3` fingerprint hash, and packet-size variance without decrypting payloads.

---

## 9. THREAT DETECTION MODULES

---

### 9.1 DDoS Detector (backend/app/detection/ddos.py)

- Purpose: Detects high-volume SYN floods, UDP floods, and amplification attacks.
- Features: `flow_packets_per_sec`, `tcp_syn_ack_ratio`, `directional_packet_ratio`, `source_entropy`.
- Logic: Layer A statistical scoring combines arrival rate intensity and unanswered SYN ratios. Layer B evaluates `ddos_rf_unsw_nb15_v1.joblib`.
- Hybrid Score:  $\max(\text{Stat\_Score}, 0.50 * \text{Stat\_Score} + 0.50 * \text{ML\_Score})$ . Alert threshold:  $\geq 0.60$ .
- Demo Status: Verified in `scripts/run_demo.py --scenario ddos`.

### 9.2 C2 Beaconsing Detector (backend/app/detection/c2.py)

- Purpose: Identifies automated low-and-slow C2 heartbeats.
- Features: Inter-arrival timestamp sequence (`delta_t`), destination IP recurrence, TLS metadata.
- Logic: Calculates temporal periodicity index  $P = 1.0 - \min(1.0, \text{std}(\text{delta\_t}) / \text{mean}(\text{delta\_t}))$ .
- Alert Threshold:  $P \geq 0.75$  and recurrence count  $\geq 4$ . Mapped to HIGH severity.
- Demo Status: Verified in `scripts/run_demo.py --scenario c2`.

### 9.3 DGA Detector (backend/app/detection/dga.py)

- Purpose: Detects Algorithmically Generated Domains used by malware for C2 rendezvous.
- Features: domain\_name, dns\_entropy, dns\_query\_length, dns\_digit\_ratio, dns\_unique\_character\_ratio.
- Logic: Evaluates Shannon entropy ( $H > 3.8$ ), label length ( $L > 18$ ), and digit concentration.
- Alert Threshold: Fused DGA score  $\geq 0.65$ . Mapped to HIGH severity.
- Demo Status: Verified in scripts/run\_demo.py --scenario dga.

#### 9.4 DNS Tunnelling Detector (backend/app/detection/dns\_tunneling.py)

- Purpose: Detects data exfiltration encoded within DNS queries.
- Features: Subdomain length, query frequency per apex domain, byte volume over UDP port 53.
- Logic: Evaluates total character volume across subdomains sharing the same parent domain.
- Alert Threshold: Subdomain length  $> 30$  and query volume  $> 1000$  bytes.
- Demo Status: Verified in scripts/run\_demo.py --scenario dns\_tunnel.

#### 9.5 Encrypted Malware / TLS Detector (backend/app/detection/tls.py)

- Purpose: Identifies suspicious encrypted TLS/QUIC sessions without payload decryption.
- Features: tls\_version, tls\_ja3, tls\_std\_packet\_size, tls\_mean\_packet\_size.
- Logic: Flags SSLv3/TLS 1.0 usage, known malicious JA3 signatures, and near-zero packet-size variance ( $\text{std} < 2.0$ ).
- Alert Threshold: Mapped to MEDIUM or HIGH severity.
- Demo Status: Verified in scripts/run\_demo.py --scenario tls\_malware.

#### 9.6 Reconnaissance Detector (backend/app/detection/recon.py)

- Purpose: Detects vertical port scans, horizontal sweeps, and SYN probing.
- Features: Source IP tracking state, unique destination ports, unique target IPs, short flow ratios.
- Logic: Bounded state tracker computes port fan-out and destination fan-out sub-scores. Layer B evaluates recon\_scan\_rf\_unsw\_nb15\_v1.joblib.
- Alert Threshold: Unique ports  $\geq 3$  or unique destinations  $\geq 3$  with fused score  $\geq 0.65$ .
- Demo Status: Verified in scripts/run\_demo.py --scenario recon.

#### 9.7 Data Exfiltration Detector (backend/app/detection/exfiltration.py)

- Purpose: Identifies unauthorized large-scale outbound data transfers.
- Features: directional\_byte\_ratio, bytes\_src\_to\_dst, bytes\_dst\_to\_src, destination rarity.
- Logic: Flags flows where directional byte ratio  $> 0.95$  and outbound volume  $> 10$  MB.
- Alert Threshold: Mapped to HIGH or CRITICAL severity.
- Demo Status: Verified in scripts/run\_demo.py --scenario exfiltration.

## 10. MACHINE LEARNING PIPELINE

---

Pipeline Flow:

## **1. Dataset Discovery: UNSW\_NB15\_training-set.csv & testing-set.csv**

---

## **2. Label Mapping: Normal -> BENIGN, DoS -> DDOS, Recon -> RECON\_SCAN**

---

## **3. Feature Extraction: 12-element numeric vector**

---

## **4. Leakage-Safe Preprocessing: StandardScaler fitted on TRAIN ONLY**

---

## **5. Imbalance Strategy: class\_weight="balanced"**

---

## **6. Model Fitting: RandomForestClassifier(n\_estimators=100, seed=42)**

---

## **7. Held-Out Evaluation: Evaluated on UNSW\_NB15\_testing-set.csv**

---

## **8. Artifact Export: joblib model binary + JSON metadata manifest**

---

## **11. UNSW-NB15 DATASET INTEGRATION**

---

Dataset Source: Australian Centre for Cyber Security (ACCS), 2015

Training File: UNSW\_NB15\_training-set.csv (175,341 raw records)

Testing File: UNSW\_NB15\_testing-set.csv (82,332 raw records)

Canonical Label Mapping:

- Normal -> BENIGN
- DoS -> DDOS (when target threat is DDOS)

- Reconnaissance -> RECON\_SCAN (when target threat is RECON\_SCAN)
- Other attack categories -> EXCLUDED to preserve binary class integrity.

## 12. DDoS MODEL RESULTS (DDOS)

Model Artifact: data/models/ddos\_rf\_unsw\_nb15\_v1.joblib

Manifest File: data/manifests/ddos\_rf\_unsw\_nb15\_v1.json

Target Class: DDOS

Evaluated Test Partition: UNSW\_NB15\_testing-set.csv (41,089 held-out test samples: 37,000 BENIGN + 4,089 DDOS)

Performance Metrics (Held-Out Test Set):

- Accuracy: 0.9856 (98.56%)
- Precision (Macro): 0.9705 (97.05%)
- Recall (Macro): 0.9632 (96.32%)
- F1 Score (Macro): 0.9668 (96.68%)
- F1 Score (Weighted): 0.9857 (98.57%)
- False Positive Rate (FPR): 0.0152 (1.52%)
- False Negative Rate (FNR): 0.0078 (0.78%)

Confusion Matrix (labels=["BENIGN", "DDOS"]):

|            |  |           |
|------------|--|-----------|
| TN: 36,439 |  | FP: 561   |
| FN: 32     |  | TP: 4,057 |

Disclaimer: These metrics represent offline evaluation on the held-out UNSW-NB15 benchmark test partition. They do not guarantee identical performance on unseen operational networks.

## 13. RECON MODEL RESULTS (RECON\_SCAN)

Model Artifact: data/models/recon\_scan\_rf\_unsw\_nb15\_v1.joblib

Manifest File: data/manifests/recon\_scan\_rf\_unsw\_nb15\_v1.json

Target Class: RECON\_SCAN

Evaluated Test Partition: UNSW\_NB15\_testing-set.csv (40,496 held-out test samples: 37,000 BENIGN + 3,496 RECON\_SCAN)

Performance Metrics (Held-Out Test Set):

- Accuracy: 0.9743 (97.43%)
- Precision (Macro): 0.8888 (88.88%)
- Recall (Macro): 0.9769 (97.69%)

- F1 Score (Macro): 0.9271 (92.71%)
- F1 Score (Weighted): 0.9756 (97.56%)
- False Positive Rate (FPR): 0.0262 (2.62%)
- False Negative Rate (FNR): 0.0200 (2.00%)

Confusion Matrix (labels=["BENIGN", "RECON\_SCAN"]):

|            |  |           |
|------------|--|-----------|
| TN: 36,031 |  | FP: 969   |
| FN: 70     |  | TP: 3,426 |

## 14. HYBRID DETECTION & RISK FUSION

The RiskFusionEngine (backend/app/risk/fusion.py) aggregates findings from all 7 detectors:

**1. Primary Threat Selection: Identifies detector with highest confidence score (S\_max).**

**2. Multi-Detector Corroboration: Increases risk score if multiple independent detectors trigger on the same flow:**

$$S_{\text{fused}} = \min(1.0, S_{\text{max}} + 0.10 * (N_{\text{active}} - 1))$$

**3. Cooldown Suppression: Suppresses identical alerts from the same source IP within a 60-second window to prevent alert fatigue.**

## 15. ALERT SYSTEM

Alert Schema Fields: alert\_id, timestamp, flow\_id, source\_ip, destination\_ip, source\_port, destination\_port, protocol, threat\_class, confidence, severity, evidence, detector\_name, model\_version, source.

Sanitized Example JSON Alert Payload:

```
{
  "alert_id": "ALT-DC54CE65",
  "timestamp": "2026-09-03T14:47:49.812Z",
  "flow_id": "DEMO-DDOS-001",
  "source_ip": "192.168.1.150",
```

```
"destination_ip": "10.0.0.5",
"source_port": 54321,
"destination_port": 80,
"protocol": "TCP",
"threat_class": "DDOS_SPOOFED_SOURCE",
"confidence": 0.7362,
"severity": "HIGH",
"evidence": {
  "statistical_score": 0.7362,
  "ml_score": 0.2267,
  "ml_label": "BENIGN",
  "ml_available": true,
  "agreement": false,
  "reasons": [
    "High-volume packet rate (40000.0 pkts/sec)",
    "Unanswered SYN flood ratio (1999.0)"
  ]
},
"detector_name": "risk_fusion_engine",
"model_version": "v1.0.0-hybrid",
"source": "controlled_demo"
}
```

## 16. DASHBOARD

The SOC Analyst Dashboard (frontend/src/pages/Dashboard.jsx) connects to ws://localhost:8000/ws/alerts for live updates.

- Live Alert Feed: Sortable, filterable real-time table of incoming alerts.
- Alert Details View: Inspects exact 12-feature vectors and evidence JSON.
- Model Performance View: Visualizes accuracy, F1 score, and 2x2 confusion matrix.

## 17. CONTROLLED DEMONSTRATION

Supported scenarios in scripts/generate\_demo\_data.py and scripts/run\_demo.py:

- ddos: Volumetric SYN flood simulation.
- c2: Low-and-slow periodic beaconing sequence.
- dga: High-entropy domain query simulation.
- dns\_tunnel: Encoded subdomain exfiltration.
- tls\_malware: Encrypted session with uniform packet size distribution.
- recon: 25-port vertical scan sequence.
- exfiltration: Asymmetric high-volume outbound transfer.
- mixed: Sequential 7-stage attack chain simulation.

## 18. TESTING AND VALIDATION

---

Command: `cd backend && pytest -q`

- Total Tests: 251
- Passed: 251
- Failed: 0
- Skipped: 1 (Optional missing local model fixture fallback test)
- Execution Time: 4.41s

## 19. PASSIVE SECURITY VERIFICATION

---

Static AST analysis (`backend/tests/test_ml_integration.py` & `test_recon.py`) verifies that the detection pipeline contains:

- 0 calls to `socket.socket()`
- 0 calls to packet transmission functions (`send`, `sendp`)
- 0 active DNS resolver invocations (`dns.resolver`)
- 0 outbound HTTP client requests (`requests.get`, `urllib.request`)
- 0 TLS decryption calls (`decrypt_tls`)

## 20. PERFORMANCE

---

- Demo Telemetry Processing Rate: 5.92 to 6.16 events/second (in-memory Python event loop).
- ML Inference Latency: < 1.5 ms per 12-feature sample (joblib + scikit-learn in memory).
- Startup Latency: < 1.2 s total startup time including model loading.

## 21. COMPETITOR ANALYSIS

---

PassiveGuard AI vs Competitors:

- Darktrace / Vectra AI: Enterprise SPAN/TAP, High Return-Path Dependency, Active Mitigation Triggered, DPI/Decryption required.
- Suricata / Snort: Inline/Passive IDS, Low Return-Path Dependency, Inline Drop, Signature-based SNI.
- Zeek (Bro): Network Telemetry, Passive Logging, Scripting Heuristics, JA3 Fingerprinting.
- PassiveGuard AI: Unidirectional Data Diode Enclave, ZERO Return-Path Dependency, DISABLED Active Mitigation (Strict Passive), Metadata-only encrypted analysis, Local Random Forest + Stat Fusion.

## 22. INNOVATION / UNIQUE SELLING PROPOSITION (USP)

---

**1. Zero-Emission Unidirectional Enclave Security: Guaranteed zero packet output.**

---

**2. Non-Intrusive Metadata Analytics: Evaluates encrypted sessions without TLS decryption keys.**

---

**3. Dual-Layer Hybrid Fusion: Combines deterministic security rules with Random Forest ML.**

---

**4. Transparent Evidence Provenance: Explains exact sub-scores and feature importances for every alert.**

---

## 23. ADVANTAGES

---

- Safe for Critical Infrastructure (Zero risk of disrupting ICS control loops).
- Lightweight & Fully Offline (No cloud APIs or external threat feeds required).
- High Explainability for SOC Operators.

## 24. LIMITATIONS

---

- Synthetic & Replayed Telemetry in Demos: Demonstrations utilize synthetic or PCAP-replayed flows.
- Dataset Generalization: UNSW-NB15 was captured in 2015; feature distributions may drift on modern 2026 enterprise traffic.
- Encrypted Payload Blindness: Cannot inspect actual encrypted text payloads if metadata exhibits no anomalous features.

## 25. FUTURE ENHANCEMENTS

---

- Hardware Data Diode integration testing (e.g., Advenica, Owl Cyber Defense).
- Implementation of probability calibration (CalibratedClassifierCV).
- Online continuous learning & concept drift monitoring.
- Integration of modern datasets (e.g., CIC-IDS2017, CSE-CIC-IDS2018).

## 26. SIH REQUIREMENT MAPPING

---

SIH Requirement | PassiveGuard AI Implementation | Status

Passive Monitoring | Read-only PCAP ingestion & flow streaming with 0 packet emission | Implemented

Unidirectional Compatibility | Session tracking without requiring bidirectional TCP handshakes | Implemented

DDoS Detection | Hybrid Statistical + ML DDoS detector | Implemented

C2 Beacon Detection | Inter-arrival time periodicity & recurrence tracking | Implemented

DGA & DNS Tunneling | Subdomain Shannon entropy & label length analysis | Implemented

Encrypted Malware Analysis | TLS metadata & packet size variance analysis | Implemented

Reconnaissance Detection | Bounded state fan-out tracking & hybrid Recon ML model | Implemented

Data Exfiltration | Directional byte asymmetry ratio analysis | Implemented

Real-time Alert Dashboard | React + WebSocket live alert streaming SOC interface | Implemented

Machine Learning Integration | Random Forest models trained on UNSW-NB15 dataset | Implemented

## 27. END-TO-END DEMO FLOW (SIH JUDGE PRESENTATION SCRIPT)

---

**1. Start FastAPI Backend: `cd backend && uvicorn app.main:app --reload --port 8000`**

---

## **2. Start React Dashboard: `cd frontend && npm run dev`**

---

**3. Open Dashboard: Navigate to `http://localhost:5173`. Show 0 alerts initially.**

---

**4. Trigger DDoS Attack Simulation: `python scripts/run_demo.py --scenario ddos`**

---

**5. Trigger Reconnaissance Scanning Simulation: `python scripts/run_demo.py --scenario recon`**

---

**6. Show Model Performance Page: Navigate to `/models`. Show 98.56% DDoS accuracy and 97.43% Recon accuracy on held-out UNSW-NB15 test set.**

---

**7. Run Pytest Suite: `cd backend && pytest -q` (Demonstrate 251 passing tests).**

---

## **28. JUDGE QUESTIONS AND ANSWERS (25 TECHNICAL Q&As;)**

---

### **1. Q: How does PassiveGuard AI work on a unidirectional data diode link?**

---

A: It operates in pure observation mode. It receives incoming packet metadata from the diode's receiving interface without ever writing to or transmitting over a network socket.

### **2. Q: How can you track TCP flows if you never see TCP ACK responses from the receiver?**

---

A: Our FlowAggregator groups packets into streaming flow objects based on observed 5-tuples and unidirectional time windows, tracking source-to-destination byte/packet counts without requiring completed 3-way handshakes.

### **3. Q: Why use Machine Learning if statistical rules already detect attacks?**

---

A: Statistical rules detect known behavioral bounds, while Random Forest ML captures non-linear feature interactions that rules might miss.

### **4. Q: How do you detect malware in encrypted TLS traffic without decrypting the payload?**

---

A: We evaluate handshake metadata (TLS version, cipher suites, SNI, JA3 hashes) and packet-size variance. Malicious C2 sessions frequently exhibit fixed packet size profiles.

### **5. Q: What dataset was used to train the ML models?**

---

A: The public UNSW-NB15 cybersecurity dataset captured by the Australian Centre for Cyber Security.

### **6. Q: Did you train and evaluate on the same data partition?**

---

A: No. We strictly preserved the official predefined benchmark boundary: UNSW\_NB15\_training-set.csv for training/validation and UNSW\_NB15\_testing-set.csv for held-out evaluation.

### **7. Q: How do you prevent data leakage during feature scaling?**

---

A: Our StandardScaler is fitted exclusively on the training partition. Test set feature vectors are transformed using the pre-fitted training parameters.

### **8. Q: Why does the DDoS demo show ML Label = BENIGN while the statistical score is HIGH?**

---

A: Synthetic demo flows differ slightly in feature distribution from real UNSW-NB15 DoS traffic. Our hybrid fusion algorithm ensures statistical security signatures are never suppressed by ML disagreement.

### **9. Q: How do you calculate DGA domain risk?**

---

A: We measure the Shannon entropy of query subdomain strings. Strings with entropy > 3.8 and high digit ratios are flagged.

## **10. Q: Can PassiveGuard AI perform active mitigation (e.g., blocking an IP address)?**

---

A: No. Active response is strictly prohibited in passive enclaves because returning packets across a data diode is physically impossible and security policy forbids automated disruption of critical control loops.

## **11. Q: How are alerts delivered to the analyst?**

---

A: Alerts are persisted to a local SQLite database (data/alerts.db) and broadcast asynchronously over WebSockets (ws://localhost:8000/ws/alerts) to the React dashboard.

## **12. Q: How does the system handle high-volume alert floods?**

---

A: The RiskFusionEngine enforces a 60-second temporal cooldown deduplication window per source IP and threat class.

## **13. Q: What happens if an ML model joblib file is missing or corrupted?**

---

A: The MLInferenceEngine catches loading exceptions safely, logs a warning, and falls back seamlessly to Layer A statistical detection.

## **14. Q: What transport protocols are supported?**

---

A: TCP, UDP, and ICMP IP traffic.

## **15. Q: How do you detect C2 beaconing?**

---

A: By tracking inter-arrival timestamps (delta\_t) across recurring destination tuples to calculate a temporal periodicity index.

## **16. Q: What is the false positive rate of your DDoS model?**

---

A: 1.52% on the held-out UNSW-NB15 test set.

## **17. Q: What is the false negative rate of your Reconnaissance model?**

---

A: 2.00% on the held-out UNSW-NB15 test set.

## **18. Q: How do you verify that your code never sends network packets?**

---

A: We execute automated AST static analysis unit tests (pytest) that scan the codebase for socket creation or packet transmission function calls.

## **19. Q: Is the system scalable to 10 Gbps links?**

---

A: The current Python prototype processes ~6.0 events/second in single-threaded Python. Production 10 Gbps deployment would require C++/DPDK packet processing.

## **20. Q: Why Random Forest instead of Deep Learning / PyTorch?**

---

A: Random Forests provide fast CPU inference (<1.5 ms), low memory footprint, resilience against feature scaling issues, and direct feature importance explainability.

## **21. Q: What is JA3 fingerprinting?**

---

A: An MD5 hash of specific TLS Client Hello parameters (TLS version, accepted ciphers, extensions, elliptic curves) used to identify client application software.

## **22. Q: How is data exfiltration detected?**

---

A: By identifying asymmetric outbound byte transfers where  $\text{Bytes\_out} / \text{Bytes\_total} > 0.95$  and total volume exceeds 10 MB.

## **23. Q: Can PassiveGuard AI run completely air-gapped?**

---

A: Yes. It relies on zero cloud APIs, zero external threat intelligence feeds, and zero internet dependencies.

## **24. Q: Where are alerts stored?**

---

A: In a local, embedded SQLite database (data/alerts.db).

## 25. Q: What makes this solution unique for SIH?

A: It is purpose-built for the constraints of unidirectional IP enclaves—combining passive metadata analysis, hybrid statistical/ML fusion, and zero-emission security.

## 29. PROJECT STATUS SUMMARY

| Component             | Implementation Status | Evidence                                                              |
|-----------------------|-----------------------|-----------------------------------------------------------------------|
| PCAP Ingestion        | COMPLETE              | backend/app/ingestion/pcap_reader.py (Passed test_ingestion.py)       |
| Flow Aggregation      | COMPLETE              | backend/app/ingestion/flow_stream.py (Passed test_ingestion.py)       |
| Feature Extraction    | COMPLETE              | backend/app/features/flow_features.py (12 Features Verified)          |
| DDoS Detector         | COMPLETE              | backend/app/detection/ddos.py (Hybrid ML Verified)                    |
| C2 Detector           | COMPLETE              | backend/app/detection/c2.py (Passed test_c2.py)                       |
| DGA Detector          | COMPLETE              | backend/app/detection/dga.py (Passed test_dga.py)                     |
| DNS Tunnel Detector   | COMPLETE              | backend/app/detection/dns_tunneling.py (Passed test_dns_tunneling.py) |
| TLS Detector          | COMPLETE              | backend/app/detection/tls.py (Passed test_tls_detector.py)            |
| Recon Detector        | COMPLETE              | backend/app/detection/recon.py (Hybrid ML Verified)                   |
| Exfiltration Detector | COMPLETE              | backend/app/detection/exfiltration.py (Passed test_exfiltration.py)   |
| Risk Fusion Engine    | COMPLETE              | backend/app/risk/fusion.py (Passed test_fusion.py)                    |
| Alert Store           | COMPLETE              | backend/app/alerts/store.py (SQLite Persistence Verified)             |
| FastAPI REST API      | COMPLETE              | backend/app/main.py & api/ (Passed test_main.py)                      |
| WebSocket Stream      | COMPLETE              | backend/app/api/websocket.py (Broadcasting Verified)                  |
| React Dashboard       | COMPLETE              | frontend/src/ (Vite + React + Recharts Verified)                      |
| ML Training Pipeline  | COMPLETE              | backend/app/ml/trainer.py (UNSW-NB15 Benchmark Trained)               |
| Test Suite            | COMPLETE              | 251/251 Tests Passed (pytest -q)                                      |
| Passive Security      | VERIFIED              | 0 Socket Calls / 0 Packet Emission (AST Verified)                     |

## 30. FINAL CONCLUSION

PassiveGuard AI delivers a technically robust, non-intrusive, and explainable AI cybersecurity solution tailored specifically for the strict constraints of unidirectional IP traffic enclaves. By pairing passive metadata feature extraction with local Random Forest machine learning and deterministic behavioral heuristics, PassiveGuard AI achieves high-accuracy threat detection without ever emitting a network packet or requiring payload decryption. All 251 automated unit and integration tests pass cleanly, confirming full technical readiness for Smart India Hackathon evaluation.

FACT CHECK & VERIFICATION AUDIT:

**1. 251 Pytest Tests Passed: Verified via pytest -q execution in backend/ (251 passed, 1 skipped, 1 warning).**

---

**2. Held-Out Test Set Metrics: Metrics for DDoS (98.56% Accuracy) and Recon (97.43% Accuracy) were directly extracted from data/manifests/ddos\_rf\_unsw\_nb15\_v1.json and data/manifests/recon\_scan\_rf\_unsw\_nb15\_v1.json.**

---

**3. Zero Network Socket Calls: Verified via AST static analysis tests in backend/tests/test\_ml\_integration.py and backend/tests/test\_recon.py.**

---

**4. UNSW-NB15 Label Mapping: Verified in backend/app/ml/datasets/unsw\_nb15.py (UNSW DoS -> DDOS, UNSW Reconnaissance -> RECON\_SCAN).**

---